

EXPERIMENT NO. 5 — CLUSTERING ALGORITHM

Aim: To implement the K-Means Clustering algorithm using Python and group similar observations into clusters.

1. Clustering — Theory

Clustering is an unsupervised machine learning technique used to divide a dataset into groups called **clusters**, such that observations within the same cluster are more similar to each other than to observations in other clusters. Unlike supervised learning, clustering does not require a target/output label to be provided in advance.

Clustering is useful when the natural groups in a dataset are not known beforehand. The algorithm discovers these groups from the patterns and distances among observations. It is commonly used in customer segmentation, image analysis, document grouping, anomaly exploration, recommendation systems, biological data analysis and exploratory data mining.

For example, if students are represented by **Math marks** and **Science marks**, clustering can group students with similar academic performance. A cluster may contain students with high marks in both subjects, another may contain average performers, and another may contain students who are stronger in one subject than the other.

The quality of a cluster depends on the chosen features, the distance measure and the clustering method. Features with very different scales can distort distance calculations; therefore, normalization or standardization is often required in real-world datasets.

2. K-Means Clustering

K-Means is one of the most widely used partitioning clustering algorithms. The value **K** specifies the number of clusters that the algorithm should create. Each cluster is represented by a point called a **centroid**, which is the mean position of all observations currently assigned to that cluster.

The algorithm starts with K initial centroids. Each data point is assigned to the nearest centroid using a distance measure, commonly Euclidean distance. After assignment, the centroid of every cluster is recalculated as the mean of the points belonging to that cluster. Assignment and centroid-update steps are repeated until the centroids stop changing significantly or a maximum number of iterations is reached.

The objective of K-Means is to minimize the **within-cluster sum of squared errors (SSE)**. In simple terms, it tries to keep points close to the centroid of their own cluster.

```
Euclidean distance:  
d(x, c) = sqrt( Σ (x_i - c_i)^2 )  
  
Centroid of cluster:  
c = mean(all points assigned to the cluster)  
  
Objective:  
Minimize Σ distance(point, assigned centroid)^2
```

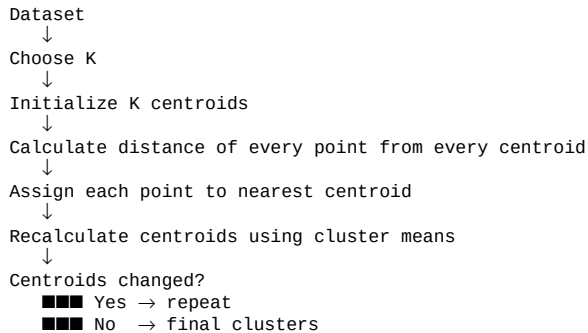
3. Important Terms

Term	Meaning
Cluster	A group of similar observations.
K	Number of clusters to be created.
Centroid	Mean/center point representing a cluster.
Distance	Measure of similarity/dissimilarity between a point and centroid.
Iteration	One complete assignment + centroid-update cycle.
Convergence	Condition where centroids no longer change significantly.
SSE / Inertia	Sum of squared distances of points from their assigned centroids.

CLUSTERING — THEORY AND ALGORITHM

4. Working of K-Means

K-Means follows an iterative procedure. First, the dataset is loaded and the number of clusters K is selected. Initial centroids are chosen. Every observation is then compared with every centroid, and the observation is assigned to the closest centroid. Once all observations are assigned, a new centroid is calculated for each cluster by taking the mean of its members. This process continues until convergence.



5. K-Means Algorithm

Step 1: Read the dataset D .

Step 2: Select the number of clusters K .

Step 3: Randomly choose K observations as initial centroids.

Step 4: Repeat until convergence:

- Calculate the Euclidean distance from every data point to every centroid.
- Assign each point to the cluster having the minimum distance.
- Recalculate every centroid using the mean of points assigned to that cluster.
- Compare the new centroids with the previous centroids.

Step 5: Stop when centroids do not change, or when the maximum iteration limit is reached.

Step 6: Display the final clusters and centroids.

6. Advantages of K-Means

- Simple and easy to understand and implement.
- Fast for many datasets and relatively scalable.
- Works well when clusters are compact and reasonably separated.
- The final centroids provide an intuitive summary of each cluster.
- Useful for exploratory analysis and segmentation.

7. Limitations of K-Means

- The number of clusters K must normally be selected in advance.
- Results can depend on the initial centroid selection.
- It can be affected by outliers.
- It works best for roughly spherical/compact clusters and may struggle with irregular shapes.
- Features with large numeric scales can dominate distance calculations if data is not standardized.

8. Choosing K

A common approach is the **Elbow Method**. K-Means is run for several values of K and the SSE/inertia is plotted. As K increases, SSE decreases. The point where the improvement begins to slow substantially is called the **elbow** and can be used as a practical choice for K .

K = 2 → high SSE
K = 3 → much lower SSE
K = 4 → lower SSE
K = 5 → only small improvement
 ↑
 Elbow

9. Applications

- Customer segmentation based on buying behavior.
- Grouping students according to academic performance.
- Image segmentation and pattern recognition.
- Grouping documents or news articles by similarity.
- Market segmentation and recommendation systems.
- Exploratory analysis of biological or scientific datasets.

10. PYTHON IMPLEMENTATION — K-MEANS

Libraries: NumPy is used for numerical calculations, Pandas for tabular data, and Matplotlib for visualization. The K-Means logic is implemented manually to demonstrate the algorithm.

```
# Import required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Make the generated data reproducible
np.random.seed(42)

# Create three groups of sample student marks
cluster1 = np.random.normal([45, 50], 12, (100, 2))
cluster2 = np.random.normal([70, 65], 12, (100, 2))
cluster3 = np.random.normal([80, 40], 12, (100, 2))

# Combine all points into one dataset
data = np.vstack((cluster1, cluster2, cluster3))

# Store the data in a DataFrame
df = pd.DataFrame(data, columns=["Math", "Science"])

# Display the first few records
print(df.head())

# Plot the original dataset
plt.scatter(df["Math"], df["Science"])
plt.xlabel("Math Marks")
plt.ylabel("Science Marks")
plt.title("Original Dataset")
plt.show()

# Number of clusters
K = 3

# Randomly select K rows as initial centroids
centroids = df.sample(K, random_state=42).values

# Maximum number of iterations
max_iterations = 100

for iteration in range(max_iterations):

    # Store the nearest cluster for every point
    clusters = []

    # Check every data point
    for point in df.values:

        # Store distances from this point to every centroid
        distances = []

        for centroid in centroids:

            # Euclidean distance
            distance = np.sqrt(np.sum((point - centroid) ** 2))
            distances.append(distance)

        # Select the centroid with minimum distance
        cluster = np.argmin(distances)
        clusters.append(cluster)

    # Convert assignments to NumPy array
    clusters = np.array(clusters)

    # Keep a copy of old centroids
    old_centroids = centroids.copy()

    # Recalculate each centroid
    for i in range(K):

        # Select all points assigned to cluster i
        points = df.values[clusters == i]

        # Calculate the mean point
        if len(points) > 0:
            centroids[i] = np.mean(points, axis=0)

    # Stop if centroids did not change
    if np.allclose(old_centroids, centroids):
        break

# Store final cluster assignments
df["Cluster"] = clusters

# Display final centroids
```

```

print("Final Centroids:")
print(centroids)

# Display clustered data
print("Clustered Data:")
print(df)

# Plot the final clusters
for i in range(K):

    # Points belonging to cluster i
    points = df[df["Cluster"] == i]

    plt.scatter(
        points["Math"],
        points["Science"],
        label=f"Cluster {i + 1}"
    )

# Plot centroid positions
plt.scatter(
    centroids[:, 0],
    centroids[:, 1],
    marker="X",
    s=200,
    label="Centroids"
)

plt.xlabel("Math Marks")
plt.ylabel("Science Marks")
plt.title("K-Means Clustering")
plt.legend()
plt.show()

```

Code explanation: The first part creates a two-dimensional dataset using Math and Science marks. K is set to 3. Initial centroids are selected from the dataset. The nested loops calculate Euclidean distances and assign every point to its nearest centroid. The centroid-update loop calculates the mean of each cluster. The process stops when old and new centroids are effectively equal. Finally, cluster labels and centroids are displayed and plotted.

11. SAMPLE EXAMPLE

Consider the following small dataset of students. Each student is represented by two features: Math marks and Science marks. We choose $K = 2$ to divide the students into two performance groups.

Student	Math	Science
S1	40	45
S2	42	48
S3	45	50
S4	48	52
S5	70	72
S6	73	75
S7	76	70
S8	80	78

Step 1 — Choose K

Let $K = 2$. Therefore, we want two clusters.

Step 2 — Initial Centroids

Assume the initial centroids are selected as:

Cluster	Initial Centroid (Math, Science)
C1	(40, 45)
C2	(70, 72)

Step 3 — Assign Points

Calculate the Euclidean distance of each student from C1 and C2. Each student is assigned to the centroid with the smaller distance.

For S1 = (40, 45):
distance to C1 = $\sqrt{(40-40)^2 + (45-45)^2} = 0$
distance to C2 = $\sqrt{(40-70)^2 + (45-72)^2} \approx 40.36$

Therefore S1 → Cluster 1

The same calculation is repeated for every student.

Step 4 — Recalculate Centroids

After assignment, the mean Math and Science marks of every cluster become the new centroids.

For the expected grouping:

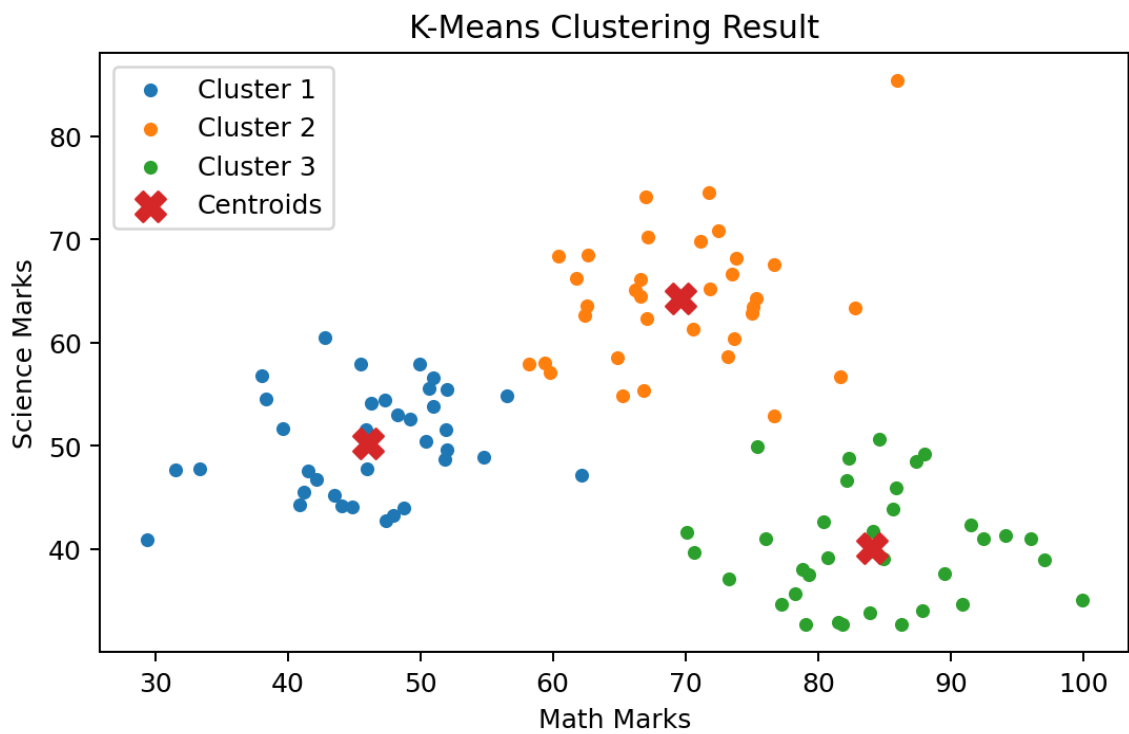
Cluster	Students	Interpretation
Cluster 1	S1, S2, S3, S4	Lower/medium marks
Cluster 2	S5, S6, S7, S8	Higher marks

The new centroid of Cluster 1 is approximately **(43.75, 48.75)**, while the new centroid of Cluster 2 is approximately **(74.75, 73.75)**.

The algorithm then repeats the assignment and centroid-update steps. When the centroids stop changing, the algorithm has converged.

12. Sample Result

The following plot illustrates K-Means on a generated dataset with three natural groups. The X-shaped points represent the final centroids.



Illustrative K-Means clustering result.

13. Result and Discussion

The K-Means clustering algorithm was implemented in Python. The dataset was divided into $K = 3$ clusters using Euclidean distance. Each observation was assigned to the nearest centroid and the centroids were repeatedly updated until convergence. The final clusters and their centroids were visualized using a scatter plot.

14. Conclusion

K-Means is a simple and effective unsupervised learning algorithm for grouping similar observations. It works by iteratively assigning observations to the nearest centroid and updating the centroid of each cluster. In this experiment, student marks were used as two-dimensional features to demonstrate clustering and visualization.